Problem Solving Using C — Question Bank & Answers

Exam Study Guide & Model Answers

Exam Guidance Note: These concise model answers are structured for maximum clarity during revision. When drawing flowcharts in your examination booklet, strictly use standard ANSI symbols: Oval (Start/Stop), Parallelogram (Input/Output), Rectangle (Process/Calculation), Diamond (Decision), and Directed Arrows. Where modern C standards specify expressions as undefined or implementation-dependent, this is explicitly highlighted rather than relying on compiler-specific assumptions.

MODULE 1 — BASICS OF C PROGRAMMING

Q1. Define a computer and explain its characteristics with an example.

A computer is an electronic, programmable device that accepts raw data as input, processes it according to stored instructions, produces meaningful information as output, and stores the results for future reference.

Core Characteristics:

- **Speed:** Executes millions of instructions per second (MIPS).
- **Accuracy:** Delivers high precision without calculation errors (Garbage In, Garbage Out principle).
- **Diligence:** Operates continuously without fatigue or loss of concentration.

- **Storage Capacity:** Stores vast quantities of data in primary and secondary memory.
- **Automation:** Performs complex tasks automatically once programmed.
- **Versatility:** Capable of handling diverse tasks ranging from mathematical calculations to graphics rendering.
- **Reliability:** Consistent performance with extremely low hardware failure rates.

Example: A college management system accepts student raw marks, calculates percentages using predefined algorithms, determines grades, and generates formatted mark sheets.

Q2. Explain the basic operations of a computer system.

A computer system performs five fundamental functional operations:

1. **Input:** Accepts raw data and instructions from input devices (e.g., keyboard, scanner).
2. **Storage:** Retains data and instructions in memory (RAM for temporary execution, Hard Drive/SSD for permanent storage).
3. **Processing:** Performs arithmetic operations (addition, subtraction) and logical comparisons using the Central Processing Unit (CPU).
4. **Output:** Converts processed binary results into human-readable information via display devices or printers.
5. **Control:** The Control Unit (CU) coordinates, directs, and sequences all system components during program execution.

Q3. Explain the generations of computers.

Generation	Circuitry / Major Technology	Primary Characteristics
1st Gen	Vacuum Tubes	Extremely large, generated massive heat, high failure rate, machine language.
2nd Gen	Transistors	Smaller, faster, more energy-efficient; introduced assembly language.
3rd Gen	Integrated Circuits (ICs)	Drastic reduction in size, higher processing speeds, keyboard & OS integration.
4th Gen	Microprocessors (VLSI/ULSI)	Personal Computers (PCs), desktop systems, high computing density, networking capabilities.
5th Gen	Ultra Large Scale Integration / AI	Parallel execution, artificial intelligence, natural language recognition.

Q4. Compare supercomputers, mainframes, minicomputers, and microcomputers.

System Type	Processing Power & Speed	Scale of Simultaneous Users	Typical Applications
Supercomputer	Highest available (Trillions of FLOPS)	Dedicated/Specialized multi-node processing	Weather forecasting, nuclear modeling, molecular simulation

System Type	Processing Power & Speed	Scale of Simultaneous Users	Typical Applications
Mainframe	Extremely High transaction rate	Thousands of concurrent enterprise users	Core banking networks, airline reservations, census databases
Minicomputer	Mid-range departmental power	Hundreds of simultaneous users	Industrial automation, research facility management
Microcomputer	Standard consumer-level processing	Single user at a time	Desktop workstations, personal laptops, educational setups

---

Q5. Differentiate between system software and application software.

Feature	System Software	Application Software
Primary Function	Controls hardware and provides a execution platform.	Solves specific user problems or business tasks.
Dependency	Runs independently of application programs.	Cannot run without an operating system present.
Interaction	User interacts with it indirectly.	Direct end-user interaction interface.
Examples	Operating Systems (Linux, Windows), Compilers, Device Drivers.	Web browsers, Word processors, Hospital management systems.

Q6. Hospital patient-record software: Identify the category and justify.

Category: Special-Purpose / Customized Application Software.

Justification: This software is explicitly developed to handle hospital-specific administrative and clinical workflows—including patient admission logs, electronic health records (EHR), doctor scheduling, medical billing, and access control permissions. General-purpose packages (such as standard spreadsheets) lack the strict security compliance, structured patient data integrity, and specialized clinical features required.

Q7. Define translator; differentiate assembler, compiler, and interpreter.

A translator is a system program that converts program code written in one programming language (source code) into an equivalent code in another language (usually low-level object or machine code).

Feature	Assembler	Compiler	Interpreter
Input Source	Assembly language (mnemonics)	High-level language	High-level language
Translation Mode	Converts mnemonics directly to machine code	Translates the **entire** source code into object file before execution	Translates and executes code **line-by-line** at runtime
Execution Speed	Very fast	Fast (post-compilation execution)	Slower (runtime translation overhead)
Error Handling	Basic mnemonic error reporting	Displays all syntax errors together after scanning entire file	Halts execution immediately at the first line containing an error

---

Q8. Steps to design an efficient program, with an example.

1. **Problem Definition:** Clearly specify requirements, constraints, and objectives.
2. **Input/Output Analysis:** Determine expected inputs and required output formats.
3. **Algorithm Design:** Develop a step-by-step logical sequence for processing.
4. **Flowchart/Pseudocode:** Visually chart or draft structured pseudo-instructions.
5. **Coding:** Write syntax-compliant source code in the chosen language (e.g., C).
6. **Compilation:** Translate source code to machine code and fix syntax errors.
7. **Testing:** Test program execution using normal, boundary, and edge-case test data.
8. **Debugging:** Detect and fix logical and runtime errors.
9. **Documentation & Maintenance:** Add comments, compile user guides, and perform periodic updates.

Example: Calculate sum of two numbers:

- Inputs: a, b
- Process: sum = a + b
- Output: sum

Q9. Define algorithm; state its advantages and disadvantages.

An algorithm is a well-defined, finite sequence of unambiguous, step-by-step instructions designed to solve a specific computational problem.

- **Advantages:**
  - Independent of any specific programming language syntax.
  - Facilitates systematic problem breakdown, logical planning, and testing.
  - Simplifies modular debugging before writing actual code.
- **Disadvantages:**
  - Writing algorithms for complex or heavily nested problems can be time-consuming.
  - Cannot be executed directly by a computer hardware system.

Q10. Algorithm and Flowchart outline: Even or Odd number.

Algorithm:

1. Start
2. Input integer `n`
3. If `n % 2 == 0`, then display "Even"
4. Else, display "Odd"
5. Stop

Flowchart Representation:

- `[Start]` (Oval) $\rightarrow$ `[/Read n/]` (Parallelogram) $\rightarrow$ `<Is n % 2 == 0?>` (Diamond)
- **Yes Branch:** `[/Print "Even"/]` (Parallelogram) $\rightarrow$ `[Stop]` (Oval)
- **No Branch:** `[/Print "Odd"/]` (Parallelogram) $\rightarrow$ `[Stop]` (Oval)

Q11. Algorithm and Flowchart outline: Find largest of three numbers.

Algorithm:

1. Start
2. Input three numbers `a`, `b`, and `c`
3. If `a >= b` and `a >= c`, then set `max = a`
4. Else if `b >= a` and `b >= c`, then set `max = b`
5. Else, set `max = c`
6. Print `max`
7. Stop

Q12. Electricity bill calculation algorithm and slab logic.

Slab Rules:

- First 100 units: ₹3.50 per unit
- Next 200 units (101–300): ₹4.50 per unit
- Above 300 units: ₹6.00 per unit

Algorithm:

1. Start
2. Read consumer units `u`
3. If `u <= 100`, then `bill = u * 3.50`
4. Else if `u <= 300`, then `bill = (100 * 3.50) + ((u - 100) * 4.50)`
5. Else, `bill = (100 * 3.50) + (200 * 4.50) + ((u - 300) * 6.00)`
6. Print `bill`
7. Stop

Q13. Explain six standard sections of a C program with an example.

1. **Documentation Section:** Contains comments describing program purpose, author, and date (`/* ... */`).
2. **Link Section:** Contains header inclusions (`#include <stdio.h>`) required for library function access.
3. **Definition Section:** Defines symbolic constants (`#define MAX 100`).
4. **Global Declaration Section:** Declares global variables and function prototypes (`int add(int, int);`).
5. **Main Function Section (`main()`):** The entry point of execution containing declarations and execution statements.
6. **Subprogram / User-Defined Functions Section:** Code implementations for defined user functions.

```
/* Documentation Section: Program to demonstrate structure */

#include <stdio.h>       /* Link Section */

#define MSG "Sum"        /* Definition Section */
```

```
int add(int, int);       /* Global Declaration Section */

int main(void) {         /* Main Function Section */

    int result = add(5, 7);

    printf("%s = %d\n", MSG, result);

    return 0;

}

int add(int x, int y) {  /* User-Defined Function Section */

    return x + y;

}
```

Q14. Describe five types of files used in C programming.

1. **Source File (`.c`):** Contains user-written, human-readable C source statements.
2. **Header File (`.h`):** Contains function declarations, macro definitions, and system definitions (e.g., `stdio.h`).
3. **Object File (`.o` / `.obj`):** Unlinked binary machine code produced by the compiler.
4. **Executable File (`.exe` / binary):** Fully linked binary code generated by the linker, directly runnable by OS.
5. **Library File (`.a` / `.lib` / `.so`):** Precompiled collections of object files containing reusable module logic.

Q15. Explain compilation and execution steps with a sequence diagram.

Compilation Pipeline:

1. **Preprocessor:** Processes `#include` statements and expands `#define` macros.
2. **Compiler:** Translates expanded code into low-level assembly / object code (`.obj`).
3. **Assembler:** Converts assembly statements into binary relocatable machine instructions.
4. **Linker:** Combines object code with system library object files to build a standalone executable file.
5. **Loader / OS:** Loads executable binary from storage into primary memory (RAM) and starts instruction execution.

Q16. C Identifier rules; classify example identifiers.

Naming Rules:

- Must start with a letter (`a-z`, `A-Z`) or an underscore (`_`).
- Subsequent characters may contain letters, digits (`0-9`), and underscores.
- White spaces, hyphens, and special symbols (except `_`) are strictly prohibited.
- C keywords cannot be used as identifier names.
- Identifiers are case-sensitive (`val` $\neq$ `Val`).

Identifier Classification Table:

Identifier	Validity	Reason
`2ndvalue`	**Invalid**	Cannot begin with a numerical digit
`emp_name`	**Valid**	Uses valid alphabetic characters and underscore

Identifier	Validity	Reason
`float`	**Invalid**	Reserved keyword in C
`my__name`	**Valid**	Consecutive underscores are valid
`basic-pay`	**Invalid**	Contains illegal hyphen operator symbol
`_total`	**Valid**	Starts with an underscore

Q17. Classify C data types; list primary types, typical sizes, and format specifiers.

Categories:

- **Basic / Primary:** `char`, `int`, `float`, `double`, `void`
- **Derived:** Arrays, Pointers, Functions
- **User-Defined:** `struct`, `union`, `enum`, `typedef`

Data Type	Typical Size (32/64-bit Systems)	Format Specifier	Typical Range
`char`	1 Byte	`%c`	-128 to 127
`unsigned char`	1 Byte	`%c`	0 to 255
`int`	4 Bytes	`%d or %i`	-2,147,483,648 to +2,147,483,647
`unsigned int`	4 Bytes	`%u`	0 to 4,294,967,295

Data Type	Typical Size (32/64-bit Systems)	Format Specifier	Typical Range
`float`	4 Bytes	`%f`	3.4×10^{-38} to $3.4 \times 10^{+38}$ (6-7 precision digits)
`double`	8 Bytes	`%lf`	1.7×10^{-308} to $1.7 \times 10^{+308}$ (15-17 precision digits)

Q18. Define a variable and explain declaration syntax.

A **variable** is a named location in memory used to store data values that can be modified during program execution.

Declaration Syntax:
`data_type variable_name;` or `data_type var1 = value1, var2;`int age = 20;

float salary = 45000.50f;

char grade = 'A';

Q19. Types of constants and declaration methods.

- **Integer Constants:** Whole numbers without decimal places (e.g., `100`, `-45`).
- **Floating-point Constants:** Real numbers with decimal points or exponent forms (e.g., `3.1415`, `1.5e-3`).
- **Character Constants:** Single characters bounded within single quotes (e.g., `'Z'`, `'7'`).
- **String Literals:** Sequence of characters bounded within double quotes (e.g., `"Hello C"`).

Declaration Methods:

1. Preprocessor macro: `#define PI 3.14159`
2. `const` keyword qualifier: `const double PI = 3.14159;`

Q20. Define operator, operand, expression; explain six C operator types.

- **Operand:** A data item (constant or variable) on which an operation is executed.
- **Operator:** A symbol that instructs the compiler to perform a specific mathematical or logical action.
- **Expression:** A combination of operands and operators that evaluates to a single output value (e.g., `a + b * 2`).

Types of Operators:

1. **Arithmetic:** Performs arithmetic computations (`+`, `-`, `*`, `/`, `%`).
2. **Relational:** Compares two scalar values (`<`, `>`, `<=`, `>=`, `==`, `!=`).
3. **Logical:** Combines condition evaluations (`&&`, `||`, `!`).
4. **Assignment:** Assigns right-hand expression values to left-hand memory variables (`=`, `+=`, `-=`, `*=`, `/=`).
5. **Increment/Decrement:** Unary operations modifying scalar value by 1 (`++`, `--`).
6. **Bitwise:** Operates directly on binary bit representation (`&`, `|`, `^`, `~`, `<<`, `>>`).

Q21. Prefix vs. Postfix increment and decrement operations (`x = 10`).

Given initial state `x = 10` independently evaluated for each expression:

Expression	Evaluated Value Assigned to y	Final Value of x	Explanation
`y = x++;`	`10`	`11`	**Postfix:** Assigns current `x` to `y`, then increments `x`.
`y = ++x;`	`11`	`11`	**Prefix:** Increments `x` first, then assigns updated value to `y`.
`y = x--;`	`10`	`9`	**Postfix:** Assigns current `x` to `y`, then decrements `x`.
`y = --x;`	`9`	`9`	**Prefix:** Decrements `x` first, then assigns updated value to `y`.

Q22. Operator precedence and associativity evaluation.

- **Precedence:** Determines which operator is evaluated first in an expression containing different operators.
- **Associativity:** Specifies the direction (Left-to-Right or Right-to-Left) in which equal-precedence operators are bound.

Evaluation of Expression: `C = 2 + 3 * 6 + 7 - 4 % 2`

1. Higher precedence operators (`*`, `%`) evaluate Left-to-Right:
    - `3 * 6` $\rightarrow$ `18`
    - `4 % 2` $\rightarrow$ `0`
    - Expression becomes: `C = 2 + 18 + 7 - 0`
2. Addition and Subtraction evaluate Left-to-Right:
    - `2 + 18` $\rightarrow$ `20`
    - `20 + 7` $\rightarrow$ `27`
    - `27 - 0` $\rightarrow$ `27`

3. **Final Result:** `C = 27`

Q23. Bitwise operations on `a = 12` and `b = 10`.

Binary representations (8-bit):
`a = 12 = 0000 1100`
`b = 10 = 0000 1010`

- **Bitwise AND (`a & b`):** `0000 1000` $\rightarrow$ **8**
- **Bitwise OR (`a | b`):** `0000 1110` $\rightarrow$ **14**
- **Bitwise XOR (`a ^ b`):** `0000 0110` $\rightarrow$ **6**
- **Left Shift (`a << 2`):** `0011 0000` $\rightarrow$ **48**
- **Right Shift (`a >> 2`):** `0000 0011` $\rightarrow$ **3**
- **Bitwise NOT (`~a`):** Flips all bits. On a standard 32-bit two's complement signed integer system, `~12` evaluates to **-13**.

Q24. Ternary operator syntax and C program to find the largest of three numbers.

Syntax: `condition ? expression_if_true : expression_if_false;`#include <stdio.h>

int main(void) {

int a, b, c, max;

printf("Enter three integers: ");

if (scanf("%d %d %d", &a, &b, &c) == 3) {

max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);

```
        printf("Largest = %d\n", max);
    }
    return 0;
}
```

Q25. Evaluate expression: `x = (++a, b += a)` where `a = 2, b = 3, x = 0`.

- The comma operator `,` evaluates operands Left-to-Right and returns the value of the rightmost operand.

1. `++a` executes: `a` increments from `2` to `3`.
2. `b += a` executes: `b = 3 + 3 = 6`.
3. The rightmost expression evaluates to `6`.
4. Result assigned to `x`: **`x = 6`**, **`a = 3`**, **`b = 6`**.

Q26. Standard I/O functions: `printf()`, `scanf()`, and conversion specifications.

- **`printf()`:** Writes formatted data output to the standard console display.
  - *Syntax:* `printf("Format String", arg1, arg2, ...);`
- **`scanf()`:** Reads formatted input data from the standard keyboard.
  - *Syntax:* `scanf("Format String", &var1, &var2, ...);`

Common Specifiers: `%d` (int), `%f` (float), `%lf` (double), `%c` (char), `%s` (string), `%u` (unsigned int).

Q27. Implicit type conversion vs. explicit type casting.

- **Implicit Conversion (Type Promotion):** Done automatically by the C compiler to prevent precision loss when evaluating mixed-type expressions.
    - *Example:* `double result = 5 + 2.5;` (The integer `5` is automatically promoted to `5.0`).
- **Explicit Casting:** Manual type conversion requested explicitly by programmer using type syntax `(data_type)`.
    - *Example:* `float average = (float)sum / count;` (Forces floating-point division instead of integer truncation).

Q28. Program to compute Gross Salary.

Rules: DA = 70% of Basic, HRA = 20% of Basic, PF = 12% of Basic. Gross = Basic + DA + HRA - PF.#include <stdio.h>

int main(void) {

double basic, da, hra, pf, gross;

printf("Enter Basic Salary: ");

if (scanf("%lf", &basic) == 1) {

da = 0.70 * basic;

hra = 0.20 * basic;

pf = 0.12 * basic;

gross = basic + da + hra - pf;

printf("Basic: %.2f\nDA: %.2f\nHRA: %.2f\nPF: %.2f\nGross Salary: %.2f\n",

basic, da, hra, pf, gross);

```
    }

    return 0;

}
```

Q29. Evaluate the eight C arithmetic and assignment expressions.

1. `a=5, b=2, c=3`: `a + b * c - a / b` $\rightarrow$ `5 + (2*3) - (5/2)` $\rightarrow$ `5 + 6 - 2 = 9` *(integer division truncates 5/2 to 2)*.
2. `x=10; y = x++ + ++x`: **Undefined Behavior** in C standards (multiple unsequenced modifications of `x` within a single sequence point).
3. `a=5; b = a++ + a++`: **Undefined Behavior** (unsequenced modification of `a`).
4. `a=2, b=3, c=4`: `a > b ? a : b > c ? b : c` $\rightarrow$ `2 > 3 ? 2 : (3 > 4 ? 3 : 4)` $\rightarrow$ **`4`**.
5. `a=5, b=10, c=15`: `a < b && b < c || a > c` $\rightarrow$ `(5 < 10 && 10 < 15) || 5 > 15` $\rightarrow$ `(1 && 1) || 0` $\rightarrow$ **`1` (True)**.
6. `a=4, b=5`: `a & b == 5` $\rightarrow$ Relational `==` binds tighter than Bitwise `&`. Evaluates as `a & (b == 5)` $\rightarrow$ `4 & (5 == 5)` $\rightarrow$ `4 & 1` $\rightarrow$ `0100 & 0001` $\rightarrow$ **`0`**.
7. `x=5; y = -x++`: Postfix `++` evaluates value `5` for assignment, negated to `-5`. Assigned to `y`: **`y = -5`**, then `x` increments to **`x = 6`**.
8. `d = a > b ? a : c > d ? c : d`: Reading uninitialized variable `d` within its own condition causes **Undefined / Indeterminate Behavior**.

Q30. Flowchart outline tasks.

(For drawing flowcharts, map each task to standard ANSI symbols: Oval start $\rightarrow$ Input parallelogram $\rightarrow$ Processing rectangle $\rightarrow$ Decision diamond $\rightarrow$ Output parallelogram $\rightarrow$ End oval).

Q31. Basic C Program Code Models.

31.1 Print Name and Welcome Message

```
#include <stdio.h>

int main(void) {

    printf("Dhanush Gangadhar\nWelcome to C Programming!\n");

    return 0;

}
```

31.2 Read Two Integers and Display Sum

```
#include <stdio.h>

int main(void) {

    int a, b;

    if (scanf("%d %d", &a, &b) == 2) {

        printf("Sum = %d\n", a + b);

    }

    return 0;

}
```

31.3 Compute Difference, Product, Quotient, and Remainder

```
#include <stdio.h>
```

```
int main(void) {
    int a, b;
    if (scanf("%d %d", &a, &b) == 2) {
        printf("Difference = %d\nProduct = %d\n", a - b, a * b);
        if (b != 0) {
            printf("Quotient = %d\nRemainder = %d\n", a / b, a % b);
        } else {
            printf("Division by zero error.\n");
        }
    }
    return 0;
}
```

31.4 Area and Perimeter of Rectangle

```
#include <stdio.h>
int main(void) {
    float length, width;
    if (scanf("%f %f", &length, &width) == 2) {
        printf("Area = %.2f\nPerimeter = %.2f\n",
               length * width, 2 * (length + width));
```

```
    }

    return 0;

}
```

31.5 Area and Perimeter of Square

```
#include <stdio.h>

int main(void) {

    float side;

    if (scanf("%f", &side) == 1) {

        printf("Area = %.2f\nPerimeter = %.2f\n", side * side, 4 * side);

    }

    return 0;

}
```

31.6 Area of Triangle (Base & Height)

```
#include <stdio.h>

int main(void) {

    float base, height;

    if (scanf("%f %f", &base, &height) == 2) {

        printf("Area = %.2f\n", 0.5f * base * height);

    }
```

```
    return 0;
}
```

31.7 Area of Circle

```
#include <stdio.h>

int main(void) {
    float radius;
    if (scanf("%f", &radius) == 1) {
        printf("Area = %.2f\n", 3.14159f * radius * radius);
    }
    return 0;
}
```

31.8 Convert Celsius to Fahrenheit

```
#include <stdio.h>

int main(void) {
    float celsius;
    if (scanf("%f", &celsius) == 1) {
        printf("Fahrenheit = %.2f\n", (celsius * 9.0f / 5.0f) + 32.0f);
    }
    return 0;
```

```
}
```

31.9 Swap Two Numbers Using Temporary Variable

```
#include <stdio.h>

int main(void) {

    int a, b, temp;

    if (scanf("%d %d", &a, &b) == 2) {

        temp = a;

        a = b;

        b = temp;

        printf("After Swap: a = %d, b = %d\n", a, b);

    }

    return 0;

}
```

31.10 Swap Two Numbers Without Temporary Variable

```
#include <stdio.h>

int main(void) {

    int a, b;

    if (scanf("%d %d", &a, &b) == 2) {

        a = a + b;
```

```
        b = a - b;
        a = a - b;
        printf("After Swap: a = %d, b = %d\n", a, b);
    }
    return 0;
}
```

31.11 Area of Triangle Using Heron's Formula

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double a, b, c, s, area;
    if (scanf("%lf %lf %lf", &a, &b, &c) == 3) {
        s = (a + b + c) / 2.0;
        area = sqrt(s * (s - a) * (s - b) * (s - c));
        printf("Area = %.2f\n", area);
    }
    return 0;
}
```

31.12 Calculate Simple Interest

```
#include <stdio.h>

int main(void) {

    float p, r, t, si;

    if (scanf("%f %f %f", &p, &r, &t) == 3) {

        si = (p * r * t) / 100.0f;

        printf("Simple Interest = %.2f\n", si);

    }

    return 0;

}
```

31.13 Convert Fahrenheit to Celsius

```
#include <stdio.h>

int main(void) {

    float fahrenheit;

    if (scanf("%f", &fahrenheit) == 1) {

        printf("Celsius = %.2f\n", (fahrenheit - 32.0f) * 5.0f / 9.0f);

    }

    return 0;

}
```

MODULE 2 — CONDITIONAL STATEMENTS & LOOPS

Q1. Define decision control and explain its need.

Decision control statements alter the sequential execution flow of a program based on the evaluation of specified relational or logical conditions.

Need: Real-world computations require dynamic paths based on input data (e.g., verifying user authorization, checking marks for pass/fail, or computing variable taxes).

Q2. Conditional branching statements in C.

1. **`if` Statement:** Executes a code block only if condition evaluates to true (non-zero).
2. **`if-else` Statement:** Selects between two distinct code blocks.
3. **`else-if` Ladder:** Tests multiple sequential conditions in order.
4. **Nested `if`:** Places an `if` construct inside another `if` or `else` branch.
5. **`switch` Statement:** Multi-way branch selection matching integral expression against constant cases.
6. **Conditional Operator (`?:`):** Inline shorthand expression for two-way branching.

Q3. Explain `if` and `if-else` with syntax, flow logic, and example.

/* Syntax */

if (condition) {

```
    /* True Branch Statements */
} else {
    /* False Branch Statements */
}
```

```
#include <stdio.h>
int main(void) {
    int num;
    if (scanf("%d", &num) == 1) {
        if (num % 2 == 0) {
            printf("%d is Even\n", num);
        } else {
            printf("%d is Odd\n", num);
        }
    }
    return 0;
}
```

Q4. Explain `else-if` ladder with a grade calculation program.

```
#include <stdio.h>

int main(void) {

    int marks;

    printf("Enter marks: ");

    if (scanf("%d", &marks) == 1) {

        if (marks >= 90) {

            printf("Grade A\n");

        } else if (marks >= 75) {

            printf("Grade B\n");

        } else if (marks >= 50) {

            printf("Grade C\n");

        } else {

            printf("Grade F (Fail)\n");

        }

    }

    return 0;

}
```

Q5. Nested `if`: Syntax and code demonstration.

```
#include <stdio.h>

int main(void) {

    int n;

    printf("Enter an integer: ");

    if (scanf("%d", &n) == 1) {

        if (n >= 0) {

            if (n == 0) {

                printf("Number is Zero\n");

            } else {

                printf("Number is Positive\n");

            }

        } else {

            printf("Number is Negative\n");

        }

    }

    return 0;
```

```
}
```

Q6. Explain `switch` statement syntax, roles of `break` and `default`.

- **`switch` Expression:** Evaluates an integer or character expression once.
- **`break` Statement:** Immediately exits the `switch` block. If omitted, execution falls through to subsequent case statements regardless of their labels.
- **`default` Label:** Optional fallback block that executes if no case constants match the evaluated expression.

```
#include <stdio.h>

int main(void) {

   int choice = 2;

   switch (choice) {

      case 1: printf("Option 1 Selected\n"); break;

      case 2: printf("Option 2 Selected\n"); break;

      default: printf("Invalid Selection\n");

   }

   return 0;

}
```

Q7. Six rules governing `switch` statements; fall-through behavior.

1. The `switch` expression **must** evaluate to an integer or character type.

2. `case` label values must be compile-time **integral constants**.
3. Duplicate `case` values are strictly prohibited.
4. `break` statements are syntactically optional but logically necessary to prevent fall-through.
5. The `default` block can appear anywhere within the `switch`, but is conventionally placed at the end.
6. Floating-point variables/constants cannot be used in `switch` expressions or `case` labels.

Q8. Iterative statements; counter-controlled vs. sentinel-controlled loops.

- **Counter-Controlled Loop:** Iterates a known, fixed number of times using an explicit loop counter variable (e.g., standard `for` loop from 1 to 100).
- **Sentinel-Controlled Loop:** Repeats indefinitely until a special sentinel input value (such as `-1` or `0`) is encountered by the program.

Q9. Compare `while` and `do-while` loop constructs.

Property	while Loop	do-while Loop
Control Category	Entry-Controlled	Exit-Controlled
Condition Check	Checked **before** executing body	Checked **after** executing body
Minimum Iterations	**0** (If initial condition is false)	**1** (Always runs at least once)
Syntax Format	`while(cond) { ... }`	`do { ... } while(cond);`

Q10. `for` loop header components and iteration sequence.

Header Structure: `for (initialization; condition; update)`

1. **Initialization:** Runs once before entering loop body.
2. **Condition:** Checked before each iteration. If true, body executes; if false, loop terminates.
3. **Body Execution:** Executes loop block statements.
4. **Update:** Increments/decrements counter variable after body completes, then re-checks condition.

Q11. Structured comparison of loop constructs in C.

```
/* for Loop */

for (int i = 1; i <= 5; i++) {

    printf("%d ", i);

}

/* while Loop */

int j = 1;

while (j <= 5) {

    printf("%d ", j);

    j++;

}

/* do-while Loop */
```

```
int k = 1;

do {

    printf("%d ", k);

    k++;

} while (k <= 5);
```

Q12. Jump statements in C: `break`, `continue`, `goto`, `return`.

- **`break`:** Terminates execution of the innermost enclosing loop or `switch` block immediately.
- **`continue`:** Skips the remaining statements in the current iteration and jumps directly to the loop update/condition evaluation step.
- **`goto`:** Unconditionally transfers control flow to a specified labeled statement within the same function.
- **`return`:** Exits current function execution and optionally returns a value to the caller.

Q13. Program to calculate quadratic equation roots (all discriminant cases).

$$\text{Discriminant } D = b^2 - 4ac$$#include <stdio.h>

```
#include <math.h>

int main(void) {

    double a, b, c, d, r1, r2, realPart, imagPart;
```

```
printf("Enter coefficients a, b, c: ");
if (scanf("%lf %lf %lf", &a, &b, &c) == 3) {
    if (a == 0) {
        printf("Not a quadratic equation.\n");
        return 0;
    }
    d = (b * b) - (4 * a * c);
    if (d > 0) {
        r1 = (-b + sqrt(d)) / (2 * a);
        r2 = (-b - sqrt(d)) / (2 * a);
        printf("Real & Distinct Roots: %.2f and %.2f\n", r1, r2);
    } else if (d == 0) {
        r1 = -b / (2 * a);
        printf("Real & Equal Roots: %.2f and %.2f\n", r1, r1);
    } else {
        realPart = -b / (2 * a);
        imagPart = sqrt(-d) / (2 * a);
        printf("Complex Roots: %.2f + %.2fi and %.2f - %.2fi\n",
```

```
                realPart, imagPart, realPart, imagPart);
        }
    }
    return 0;
}
```

Q14. Vowel checker program using `switch` fall-through.

```
#include <stdio.h>

int main(void) {
    char ch;
    printf("Enter a character: ");
    if (scanf(" %c", &ch) == 1) {
        switch (ch) {
            case 'a': case 'A':
            case 'e': case 'E':
            case 'i': case 'I':
            case 'o': case 'O':
            case 'u': case 'U':
```

```
            printf("%c is a Vowel\n", ch);
            break;
        default:
            printf("%c is a Consonant or non-alphabetic character\n", ch);
    }
  }
  return 0;
}
```

Q15. Repeating menu-driven arithmetic calculator.

```
#include <stdio.h>

int main(void) {
  int choice;
  double a, b;
  do {
    printf("\n--- MENU CALCULATOR ---\n");
    printf("1. Add\n2. Subtract\n3. Multiply\n4. Divide\n0. Exit\nChoice: ");
    if (scanf("%d", &choice) != 1) break;
```

```
        if (choice >= 1 && choice <= 4) {
            printf("Enter two numbers: ");
            scanf("%lf %lf", &a, &b);
        }
        switch (choice) {
            case 1: printf("Result = %.2f\n", a + b); break;
            case 2: printf("Result = %.2f\n", a - b); break;
            case 3: printf("Result = %.2f\n", a * b); break;
            case 4:
                if (b != 0) printf("Result = %.2f\n", a / b);
                else printf("Error: Division by zero!\n");
                break;
            case 0: printf("Exiting program.\n"); break;
            default: printf("Invalid choice, try again.\n");
        }
    } while (choice != 0);
    return 0;
}
```

Q16. Module 2 Practice Programs.

16.1 Reverse Digits of an Integer Using `while`

```
#include <stdio.h>

int main(void) {

    int n, rev = 0, rem;

    if (scanf("%d", &n) == 1) {

        while (n != 0) {

            rem = n % 10;

            rev = (rev * 10) + rem;

            n /= 10;

        }

        printf("Reversed Integer = %d\n", rev);

    }

    return 0;

}
```

16.2 Sum of Digits Using `for` Loop

```
#include <stdio.h>

int main(void) {
```

```
    int n, sum = 0;
    if (scanf("%d", &n) == 1) {
        for (; n != 0; n /= 10) {
            sum += n % 10;
        }
        printf("Sum of Digits = %d\n", sum);
    }
    return 0;
}
```

16.3 Sums and Counts of Even and Odd Numbers

```
#include <stdio.h>
int main(void) {
    int n, val, evenCount = 0, oddCount = 0;
    long evenSum = 0, oddSum = 0;
    if (scanf("%d", &n) == 1) {
        for (int i = 0; i < n; i++) {
            scanf("%d", &val);
            if (val % 2 == 0) {
```

```
            evenSum += val;
            evenCount++;
        } else {
            oddSum += val;
            oddCount++;
        }
    }
    printf("Even Count: %d | Even Sum: %ld\n", evenCount, evenSum);
    printf("Odd Count: %d | Odd Sum: %ld\n", oddCount, oddSum);
  }
  return 0;
}
```

16.4 Pyramid Pattern of Stars

```
#include <stdio.h>
int main(void) {
  int rows;
  if (scanf("%d", &rows) == 1) {
    for (int i = 1; i <= rows; i++) {
      for (int j = 1; j <= rows - i; j++) printf(" ");
```

```
            for (int k = 1; k <= (2 * i - 1); k++) printf("*");
            printf("\n");
        }
    }
    return 0;
}
```

16.5 Find Largest and Smallest of Three Numbers

```
#include <stdio.h>

int main(void) {
    int a, b, c, max, min;
    if (scanf("%d %d %d", &a, &b, &c) == 3) {
        max = min = a;
        if (b > max) max = b;
        if (c > max) max = c;
        if (b < min) min = b;
        if (c < min) min = c;
        printf("Largest = %d | Smallest = %d\n", max, min);
    }
```

```
    return 0;
}
```

16.6 Purchase Discount Processing

```
#include <stdio.h>

int main(void) {
    double amount, rate, discount, net;
    while (1) {
        printf("Enter bill amount (0 to exit): ");
        if (scanf("%lf", &amount) != 1 || amount == 0) break;
        if (amount < 1000) rate = 0.0;
        else if (amount < 5000) rate = 0.05;
        else if (amount < 10000) rate = 0.10;
        else rate = 0.15;
        discount = amount * rate;
        net = amount - discount;
        printf("Discount = %.2f | Net Payable = %.2f\n", discount, net);
    }
    return 0;
}
```

16.7 Check Positive, Negative, or Zero

```
#include <stdio.h>

int main(void) {

    int n;

    if (scanf("%d", &n) == 1) {

        if (n > 0) printf("Positive\n");

        else if (n < 0) printf("Negative\n");

        else printf("Zero\n");

    }

    return 0;

}
```

16.8 Check Even or Odd

```
#include <stdio.h>

int main(void) {

    int n;

    if (scanf("%d", &n) == 1) {

        printf("%s\n", (n % 2 == 0) ? "Even" : "Odd");

    }

    return 0;
```

```
}
```

16.9 Check Pass/Fail (Threshold = 40)

```
#include <stdio.h>

int main(void) {

    int marks;

    if (scanf("%d", &marks) == 1) {

        printf("%s\n", (marks >= 40) ? "Pass" : "Fail");

    }

    return 0;

}
```

16.10 Check Leap Year

```
#include <stdio.h>

int main(void) {

    int year;

    if (scanf("%d", &year) == 1) {

        if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {

            printf("%d is a Leap Year\n", year);

        } else {

            printf("%d is NOT a Leap Year\n", year);
```

```
        }
    }
    return 0;
}
```

16.11 Print Day of Week Using `switch`

```
#include <stdio.h>

int main(void) {
    int day;
    if (scanf("%d", &day) == 1) {
        switch (day) {
            case 1: printf("Monday\n"); break;
            case 2: printf("Tuesday\n"); break;
            case 3: printf("Wednesday\n"); break;
            case 4: printf("Thursday\n"); break;
            case 5: printf("Friday\n"); break;
            case 6: printf("Saturday\n"); break;
            case 7: printf("Sunday\n"); break;
            default: printf("Invalid Day Index\n");
```

```
        }
    }
    return 0;
}
```

16.12 Factorial Computation

```
#include <stdio.h>
int main(void) {
    int n;
    unsigned long long fact = 1;
    if (scanf("%d", &n) == 1) {
        if (n < 0) {
            printf("Factorial undefined for negative numbers.\n");
        } else {
            for (int i = 1; i <= n; i++) fact *= i;
            printf("Factorial of %d = %llu\n", n, fact);
        }
    }
    return 0;
}
```

16.13 Multiplication Table

```
#include <stdio.h>

int main(void) {

    int n;

    if (scanf("%d", &n) == 1) {

        for (int i = 1; i <= 10; i++) {

            printf("%d x %d = %d\n", n, i, n * i);

        }

    }

    return 0;

}
```

16.14 Check Palindrome Integer

```
#include <stdio.h>

int main(void) {

    int n, temp, rev = 0;

    if (scanf("%d", &n) == 1) {

        temp = n;

        while (temp > 0) {

            rev = (rev * 10) + (temp % 10);
```

```
        temp /= 10;
    }
    if (n >= 0 && n == rev) printf("%d is a Palindrome\n", n);
    else printf("%d is NOT a Palindrome\n", n);
  }
  return 0;
}
```

16.15 Check Prime Number

```
#include <stdio.h>

int main(void) {
  int n, isPrime = 1;
  if (scanf("%d", &n) == 1) {
    if (n < 2) isPrime = 0;
    for (int i = 2; i * i <= n; i++) {
      if (n % i == 0) {
        isPrime = 0;
        break;
      }
```

```
        }

        printf("%d is %s\n", n, isPrime ? "Prime" : "NOT Prime");

    }

    return 0;

}
```

16.16 Generate Fibonacci Series Up to n Terms

```
#include <stdio.h>

int main(void) {

    int n;

    long long a = 0, b = 1, next;

    if (scanf("%d", &n) == 1) {

        for (int i = 0; i < n; i++) {

            printf("%lld ", a);

            next = a + b;

            a = b;

            b = next;

        }

        printf("\n");

    }
```

```
    return 0;
}
```

Common C Pitfalls to Avoid in Examinations

1. **Missing Addressing Operator in `scanf`:** Forgetting `&` on scalar variable inputs (e.g., writing `scanf("%d", n)` instead of `scanf("%d", &n)`).
2. **Assignment vs. Equality Operator:** Using `=` (assignment) inside condition expressions instead of `==` (relational equality test).
3. **Integer Truncation:** Dividing integers yields integer outputs (e.g., `5 / 2` evaluates to `2`, not `2.5`).
4. **Missing Break in Switch:** Omitting `break;` statements inside `case` branches leads to unintended fall-through.
5. **Unsequenced Modifications:** Avoid writing expressions like `x++ + ++x` or `a++ + a++`, which produce undefined compiler behavior.